第六章　模板

第六章 模板

1. 模板的概念
2. 函数模板与模板函数
3. 类模板与模板类

1.模板的概念

```
int max(int a,int b)
  { return a>b?a:b;}

float max(float a,float b)
  { return a>b?a:b;}

char max(char a,char b)
  { return a>b?a:b;}
```

- 作为强类型的语言，C++ “不允许”也不应该两种不同类型的参数进行比较。一般的解决办法就是写一系列的函数，来分别完成整型、浮点型和用户自定义类型的求解。
- 可以想像，所有这些函数（的代码）几乎都是一模一样的，只是操作的类型不同。这使程序代码变得累赘而加大维护难度。

1.模板的概念

```
int max(int a,int b)
  { return a>b?a:b;}

float max(float a,float b)
  { return a>b?a:b;}

char max(char a,char b)
  { return a>b?a:b;}
```

→

```
T max(T a,T b)
  { return a>b?a:b;}
```

- C++最重要的特征之一就是代码重用，为了实现代码重用，代码必须具有通用性。通用的代码需要不受数据类型的影响，并且可以自动适应数据类型的变化。这种程序设计类型称为**参数化程序设计（泛型程序设计）**。

1.模板的概念

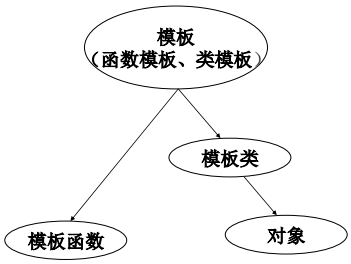
```
int max(int a,int b)
  { return a>b?a:b;}

float max(float a,float b)
  { return a>b?a:b;}

char max(char a,char b)
  { return a>b?a:b;}
```

→

```
T max(T a,T b)
  { return a>b?a:b;}
```

- C++模板是近几年来对C++的一种扩展，**模板是根据类型参数来产生函数和类的机制**，有时也称模板为“参数化的类型”。使用模板，可以设计一个对许多类型的数据进行操作的类或函数，而不需要为每个类型的数据建立一个单独的类。

1.模板的概念　　模板类型



第六章 模板

1. 模板的概念
2. 函数模板与模板函数
3. 类模板与模板类

2.函数模板与模板函数

函数模板

typename T1,typename T2,…,typename Tn
或 class T1,class T2,…,class Tn

template <类型形式参数表>
返回类型 FuctionName（形式参数表）
{
函数体
}

通常是“形式参数表”中的某一类型，也可以是其他类型

✓可以是“类型形式参数表”中声明的类型参数，也可以是普通类型

✓“类型形式参数表”中的类型参数在函数定义中至少出现一次

例：

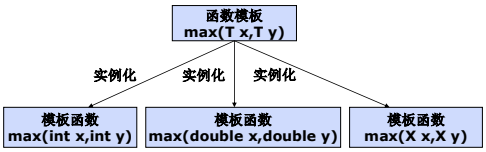
```
template <typename T>
T max(T a,T b)
  { return a>b?a:b;}
```

关键字template后的尖括号表明，max函数要用到一个叫做T的参数，这个参数是一种类型。无论参数T为int、char或其他类型(包括类类型),函数max的语意都是对x和y求最大值。

2.函数模板与模板函数

模板函数

这样定义的max代表了一类具有相同程序逻辑的函数，它不是一个真正的函数，被称为函数模板。函数模板本身是不被编译的，所以函数模板不能直接使用，必须被实例化后，也就是给定类型参数T后才能使用：



- 就像类和对象的关系一样，函数模板将具有相同正文的一类函数抽象出来，可以适应任意类型T。

2.函数模板与模板函数

模板函数

```
//函数模板
template <类型形式参数表>
返回类型 FuctionName（形式参数表）
{
  函数体
}
//模板函数
FuctionName（实参表）;
```

例：

```
//函数模板
template <typename T>
T max(T a,T b)
  { return a>b?a:b;}

void main()
 { double a = 1.0;
   double b = 2.0, c;
   c = max(a, b); //模板函数
 }
```


2.函数模板与模板函数

冒泡法排序函数模板

```
template<typename ElementType>
void SortBubble (ElementType *a, int size)
{ int i;
  ElementType temp;
for(int pass=1; pass<size; pass++)
   { for(i=0;i<size-pass;i++)
      if(a[i]>a[i+1])
       { temp = a[i];
         a[i]=a[i+1];
         a[i+1]=temp; }
   }
void main()
{ int array1[10]={55,2,6,4,32,12,9,73,26,37}
  char array2[9]={'D','F','C','A','B','G','E','I','H'}
  SortBubble(array1, 10);
  SortBubble(array2,9);}
```

模板声明

基本类型参数

声明的类型参数

定义声明的类型变量

算法实现

使用模板函数

2.函数模板与模板函数

使用多个模板参数

```
#include<iostream.h>
template<typename T>
T max (const T a, const T b)
     {return a>b?a:b;}

void main()
{  int a=6; char b='a';
   cout<<"max(a, b)is "<<max(a, b)<<endl; //eror
   cout<<"max(b, a)is "<<max(b,a)<<endl; //error
}
```

为函数模板提供了两个不同的类型（int和char），编译器无法按模板的规则实例化出那样的函数。但是int和char直接的隐式类型转换是很普遍的。

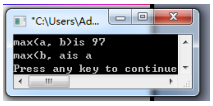
```
Compiling...
exe.cpp
i:\面向对象程序设计\实验\ch6_slide13_1\exe.cpp(8) : error C2782: 'T __cdecl max(const T,const T)' : template parameter 'T' is ambiguous
        could be 'char'
        or       'int'
i:\面向对象程序设计\实验\ch6_slide13_1\exe.cpp(9) : error C2782: 'T __cdecl max(const T,const T)' : template parameter 'T' is ambiguous
        could be 'int'
        or       'char'
Error executing cl.exe.

exe1.exe - 2 error(s), 0 warning(s)
```

2.函数模板与模板函数　使用多个模板参数

✓以上问题解决方法：一个函数模板使用多个模板参数或者重载一个函数模板。

1. 函数模板使用多个模板参数

```
#include<iostream.h>
template<typename T,typename D>
T max (const T a, const D b)
      {return a>b?a:b;}

void main()
{  int a=6; char b='a';
    cout<<"max(a, b)is "<<max(a, b)<<endl; //Ok
    cout<<"max(b, ais "<<max(b,a)<<endl; //Ok
}
```



2.函数模板与模板函数　重载函数模板

2. 重载函数模板

```
#include<iostream.h>
template<typename T>
T max (const T a, const T b)
    {return a>b?a:b;}
template<typename T>
T max (const T a, const T b,const T c)
    {T temp;temp=max(a,b);return max(temp,c);}
int max (const int a, const char b)
   {return a>b?a:b;}

void main()
{  cout<<"max(3, 'a')is "<<max(3, 'a')<<endl;
   cout<<"max(9,5, 23)is "<<max(9,5,23)<<endl;
   cout<<"max(9.3, 0.5)is "<<max(9.3, 0.5)<<endl;
}
```

函数模板

重载函数模板

用普通函数重载函数模板

2.函数模板与模板函数　重载函数模板

2. 重载函数模板

```
#include<iostream.h>
template<typename T>
T max (const T a, const T b)
    {return a>b?a:b;}
template<typename T>
T max (const T a, const T b,const T c)
    {T temp;temp=max(a,b);return max(temp,c);}
int max (const int a, const char b)
   {return a>b?a:b;}

void main()
{  cout<<"max(3, 'a')is "<<max(3, 'a')<<endl;
   cout<<"max(9,5, 23)is "<<max(9,5,23)<<endl;
   cout<<"max(9.3, 0.5)is "<<max(9.3, 0.5)<<endl;
}
```


2.函数模板与模板函数　求两数中大数完整实例

```
using namespace std;

template <typename T>
T Max( const T a, const T b )
{ return a>b ? a : b ; }

int main()
{ cout << "Max( 3, 5 ) is " << Max( 3, 5 ) << endl ;
  cout << "Max( 'y', 'e') is " << Max( 'y', 'e' ) << endl ;
  cout << "Max( 10.3, 0.5 ) is " << Max( 10.3, 0.5 ) << endl ;
}
```

2.函数模板与模板函数　求两数中大数完整实例

```
#include<iostream>
using namespace std;

template <typename T>
T Max( const T a, const T b )
{ return a>b ? a : b ; }

int main()
{ cout << "Max( 3, 5 ) is " << Max( 3, 5 ) << endl ;
  cout << "Max( 'y', 'e') is " << Max( 'y', 'e' ) << endl ;
  cout << "Max( 10.3, 0.5 ) is " << Max( 10.3, 0.5 ) << endl ;
}
```

函数模板

2.函数模板与模板函数　求两数中大数完整实例

```
#include<iostream>
using namespace std;

template <typename T>
T Max( const T a, const T b )
{ return a>b ? a : b ; }

int main()
{ cout << "Max( 3, 5 ) is " << Max( 3, 5 ) << endl ;
  cout << "Max( 'y', 'e') is " << Max( 'y', 'e' ) << endl ;
  cout << "Max( 10.3, 0.5 ) is " << Max( 10.3, 0.5 ) << endl ;
}
```

```
int max ( int a , int b )
{ return a > b ? a : b ; }
```

由实参类型实例化

2.函数模板与模板函数 求两数中大数完整实例

```
#include<iostream>
using namespace std;

template <typename T>
T Max( const T a, const T b )
{ return a>b ? a : b ; }

int max ( int a , int b )
{ return a > b ? a : b ; }

char max ( char a , char b )
{ return a > b ? a : b ; }

int main()
{ cout << "Max( 3, 5 ) is " << Max( 3, 5 ) << endl ;
  cout << "Max( 'y', 'e') is " << Max( 'y', 'e' ) << endl ;
  cout << "Max( 10.3, 0.5 ) is " << Max( 10.3, 0.5 ) << endl ;
}
```

由实参类型实例化

2.函数模板与模板函数 求两数中大数完整实例

```
#include<iostream>
using namespace std;

template <typename T>
T Max( const T a, const T b )
{ return a>b ? a : b ; }

int max ( int a , int b )
{ return a > b ? a : b ; }

char max ( char a , char b )
{ return a > b ? a : b ; }

double max ( double a , double b )
{ return a > b ? a : b ; }

int main()
{ cout << "Max( 3, 5 ) is " << Max( 3, 5 ) << endl ;
  cout << "Max( 'y', 'e') is " << Max( 'y', 'e' ) << endl ;
  cout << "Max( 10.3, 0.5 ) is " << Max( 10.3, 0.5 ) << endl ;
}
```

由实参类型实例化

2.函数模板与模板函数 求两数中大数完整实例

```
#include<iostream>
using namespace std;

template <typename T>
T Max( const T a, const T b )
{ return a>b ? a : b ; }

int max ( int a , int b )
{ return a > b ? a : b ; }

char max ( char a , char b )
{ return a > b ? a : b ; }

double max ( double a , double b )
{ return a > b ? a : b ; }

int main()
{ cout << "Max( 3, 5 ) is " << Max( 3, 5 ) << endl ;
  cout << "Max( 'y', 'e') is " << Max( 'y', 'e' ) << endl ;
  cout << "Max( 10.3, 0.5 ) is " << Max( 10.3, 0.5 ) << endl ;
}
```

编译器生成的模板函数

```
C:\WINDOWS\system32\cmd.exe
Max( 3, 5 ) is 5
Max( 'y', 'e') is y
Max( 10.3, 0.5 ) is
请按任意键继续. . .
```

程序执行时匹配不同的版本

2.函数模板与模板函数 总结（1）

➢在编译时，函数模板本身并不会被编译成实际的可执行代码，只有在函数模板被调用时（也就是模板中的类型参数有了具体的数据类型作替代时），编译器才按照实际数据类型生成新的函数，然后编译这个函数。正因为函数模板不是具体函数，如果**要将函数模板放在头文件中，必须将整个函数模板放入**，不能像普通函数那样可以只放函数的原型说明，然后将函数的具体实现放在其它文件中。

2.函数模板与模板函数 总结（2）

✓如果调用的两个参数都是同一类型，则不会发生什么问题，如果两个参数不是同一类型，就会出现错误。如：

max(100.2,5);

✓解决办法：可以使用两个类型参数的模板

```
template <class T ， class S>
T  max(T a, S b)
    {   return  (T)(a > b ? a : b);   }
```

这个函数模板的返回值类型是第一个参数的类型。

如在这个例子中 max(100,100.5);返回的是100，尽管100.5比100大，但由于第一个参数是int型的，所以double型的100.5被转换成int型。

2.函数模板与模板函数 总结（3）

✓返回值的类型是某一类型参数，则该类型参数必须同时用在函数的参数类型上。

```
template <class T >
T  Function(T a)
   {…}
```

✓如果返回类型参数与任何一个参数均不一样：

```
template<class T, class S, class U>
U FunctionA(T a, S b)
   {…}
```

则，当调用FunctionA时，C++语言编译器就会因为无法得知函数的具体返回类型而出错。

同样也不能使用未在参数表中存在的类型参数。

2.函数模板与模板函数 总结（4）

✓在template尖括号中的所有类型参数必须在函数的参数表中全部用上。

```
template <class T1 ,class T2, class T3>
T1  Function(T1 a,T2 b)   // error 未使用T3
  {…}
```

✓各个函数模板可以使用相同的符号表示类型参数，并且互不干扰。

```
Template <class T, class S>
T min(T a, S b)
  {…}

Template <class T, class S>
T max(T a, S b)
  {…}
```

在这里，min和max模板都用了符号T和S作为类型参数，但相互间没有任何实际的联系。

第六章 模板

1. 模板的概念
2. 函数模板与模板函数
3. 类模板与模板类

3.类模板与模板类 类模板和模板类定义

- 一个类模板允许用户为类定义一种模式，使得类中的某些数据成员、某些成员函数的参数或返回值，能取任意数据类型。
- 定义一个类模板与定义函数模板的格式类似，必须以关键字template开始，后面是尖括号括起来的模板参数，然后是类名：

```
template < class Type1, class Type2,…>
class 模板类名
 { //….};
```

- 类模板不代表一个具体的、实际的类，而代表着一类类。实际上，类模板的使用就是将类模板实例化成一个具体的类——模板类：

模板类名<实际的类型> 对象名；

3.类模板与模板类 类模板体内定义成员函数

```
template <类型形式参数表> class className
{   //类声明及成员函数实现 };
```

✓生成模板类对象：**className<类型实参表> object;**

```
template <class T>  //类模板声明
class pair // 类名
 {public:
   pair(T first, T second) //类成员函数
    { Value1=first;
      Value2=second;}
   private:
     T value1,value2;  //数据成员
 };
```

生成模板类并创建对象：

```
pair<int> obj1(1, 2);
```


3.类模板与模板类 类模板体外定义成员函数

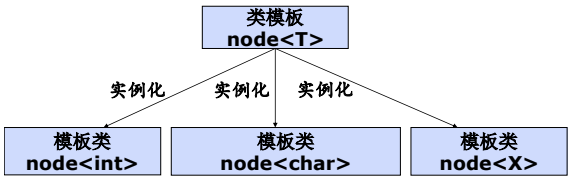
```
template <类型形式参数表>  class className  //声明类模板
{   //类声明体 };

 template <类型形式参数表>  //定义类模板成员函数
返回类型 className <类型名表> ::MemberFuctionName（形参表）
{   //成员函数定义体  }
```

```
template <class T>  //类模板声明
class pair // 类模板定义
 {public:
   pair(T first, T second);  //类成员函数
  private:
     T value1,value2;  //数据成员};

template <class T>
pair<T>::pair(T first, T second) //在类模板外定义类成员函数
   { Value1=first;
     Value2=second;}
```

若此成员函数中有模板参数存在，则需要在函数体外进行模板声明，并且在函数名前的类名后缀上“Type”。

3.类模板与模板类 类模板与模板类之间的关系

模板类是类模板对某一特定类型的实例。类模板代表了一类类，模板类表示某一具体的类：



3.类模板与模板类

使用类模板

1. 在程序的头文件中说明类模板的定义。

```
template <class T>  //类模板声明
class pair // 类名
 {public:
   pair(T first, T second) //类成员函数
    { Value1=first;
      Value2=second;}
   private:
     T value1,value2;  //数据成员
 };
```

2. 在适当的地方创建一个模板类的实例，即一个实实在在的类定义，同时创建该模板类的对象。

```
pair<int> obj1;
```

3. 有了对象名，以后的使用就和通常一样。但要记住，你规定了什么类型的模板类，在使用成员函数时，所赋得实参也要对应该类型。

3.类模板与模板类

带两个类型参数的类模板实例

```
#include<iostream.h>
template<class T1,class T2>//带两个类型参数类模板声明
class myclass
{ T1 i; T2 j;//声明类型为T1、T2的两个私有成员
  public:
    myclass(T1 a,T2 b) //带两个参数的构造函数
    { i=a;j=b;}
    void show()
    { cout<<"i= "<<i<<"  j= "<<j<<endl;}
};

void main()
  //用int和double两个类型参数实例化myclass类对象obj1
{ myclass<int,double> obj1(12,0.15);

  //用char和char *两个类型参数实例化myclass类对象obj2
  myclass<char,char *> obj2('x',"This is a test");

 obj1.show();
 obj2.show();
}
```


End